Name : - Shashank Lokesha

Mini Project : -

Reinforcement Learning for Dynamic Resource Allocation in Cloud Computing

Abstract

This project focuses on applying reinforcement learning (RL) techniques to optimize resource allocation in cloud computing environments. The objective is to dynamically allocate CPU resources to incoming tasks in order to maximize efficiency while minimizing waiting time and resource wastage. A custom simulation environment is developed, and a Q-learning agent is trained to make allocation decisions. The results demonstrate that RL-based allocation outperforms traditional methods such as random allocation.

Introduction

Cloud computing systems must efficiently manage resources like CPU and memory to handle varying workloads. Traditional static allocation strategies often lead to inefficiencies such as underutilization or congestion. Reinforcement learning offers a promising solution by enabling systems to learn optimal strategies through interaction with the environment. This project explores how RL can be applied to dynamic resource allocation.

Problem Statement

The goal of this project is to design an intelligent agent that dynamically allocates CPU resources to tasks arriving in a cloud system. The system must balance between maximizing resource utilization and minimizing task waiting time. The problem is modeled as a sequential decision-making process where the agent learns from rewards and penalties.

Methodology

A simulation-based environment is created to represent a simplified cloud system. Tasks arrive randomly and are placed in a queue. The agent observes the system state and decides how much CPU to allocate. Q-learning is used as

Proprietary

Name : - Shashank Lokesha

Mini Project : -

the learning algorithm. The agent updates its policy based on rewards received after each action.

Environment Design

The environment consists of a fixed CPU capacity and a queue of tasks with varying sizes. At each step, new tasks may arrive. The agent selects an action corresponding to a level of CPU allocation (low, medium, high). The system updates the state based on the action taken and computes the reward.

Reward Function

The reward function is designed to encourage efficient resource utilization. Positive rewards are given for completing tasks quickly, while penalties are applied for idle resources and long waiting times. Proper reward design ensures that the agent learns a balanced strategy.

Implementation

The project is implemented in Python using libraries such as NumPy and Matplotlib. The environment and agent are coded from scratch. Training is performed over multiple episodes, and performance is tracked using cumulative rewards.

Results

The RL agent shows improvement over time as it learns better allocation strategies. Compared to a random policy, the RL approach achieves higher average rewards and better resource utilization. Graphs illustrate the learning progress.

Conclusion

This project demonstrates the effectiveness of reinforcement learning in solving dynamic resource allocation problems. Even with a simplified environment, the agent learns to make intelligent decisions. Future work can include more complex environments and deep reinforcement learning techniques.

Proprietary